

Maze Escape: Human vs Monster AI Survival Game

Principles of Artificial Intelligence Project Report

Course: Principles of Artificial Intelligence

Project: Maze Escape: Human vs Monster AI Survival Game

June 26, 2026

Contents

Abstract	2
1 Introduction	2
2 Methodology	3
2.1 Genetic Algorithm for Map Generation	3
2.2 Human AI Algorithm (BFS)	5
2.3 Monster Algorithm	5
2.3.1 A* Search Algorithm	5
2.3.2 Greedy Search for Monster Movement	5
2.3.3 Simulated Annealing Algorithm	6
2.3.4 Minimax for Predictive Monster AI	8
2.4 Machine Learning Dual-Monster Mode	9
3 Game System Design and Gameplay Integration	10
3.1 Overall Game Flow	10
3.1.1 Game Execution Process	10
3.1.2 Interface Navigation and Global Button Rules	11
3.2 Game State Representation	11
3.3 Turn-Based Movement Logic	12
3.4 Interactive Item System	12
3.4.1 Swift Boots	13
3.4.2 Frost Trap	13
3.4.3 Teleport Stone	13
3.4.4 Cloak	14
3.4.5 Item-State Update and Game Balance	14
3.5 User Interface and Backend Integration	15
3.5.1 Pre-Game Pages	15
3.5.2 Gameplay Interface	16
3.6 Deployment Architecture	17
4 Validation and Evaluation	17
4.1 Map Generation and Spawn Test	17
4.2 Algorithm Evaluation	18
4.3 Machine Learning Evaluation	20
5 Conclusion	21
5.1 Results	21
5.2 Limitations	22
5.3 Future Work	23

Abstract

In order to explore the innovative application scenarios of artificial intelligence search algorithms and machine learning, and to evaluate the performance of different algorithms in specific scenarios, we created a game called maze-escape, which simulates humans being chased by monsters on a 30×30 circular grid map (packaging edge). Among them, the core driver of the monster is the search algorithm and the model trained by machine learning. The system uses and compares five artificial intelligence search strategies: A* pathfinding, greedy algorithm, breadth priority search (BFS), minimax algorithm and simulated annealing algorithm. The maze is generated by genetic algorithm (GA). GA has improved map connectivity, obstacle distribution, path diversity and branch structure through multi-generation evolution. In addition, we have also added machine learning. The trained model can be scrolled to predict the position of three human actions, thus preventing human escape.

After ensuring that there is no problem with the game logic framework and it can operate normally, we tried to evaluate the performance of different algorithms and the performance of models based on machine learning with the game as a platform. We found that the A* algorithm created the most direct pursuit path, and the greedy algorithm provided fast performance with low computing costs, and simulated retreat. The fire algorithm increases the controlled randomness, making the movement of monsters more unpredictable. The genetic algorithm constantly generates connected and navigable maps to maintain the quality of the game.

1 Introduction

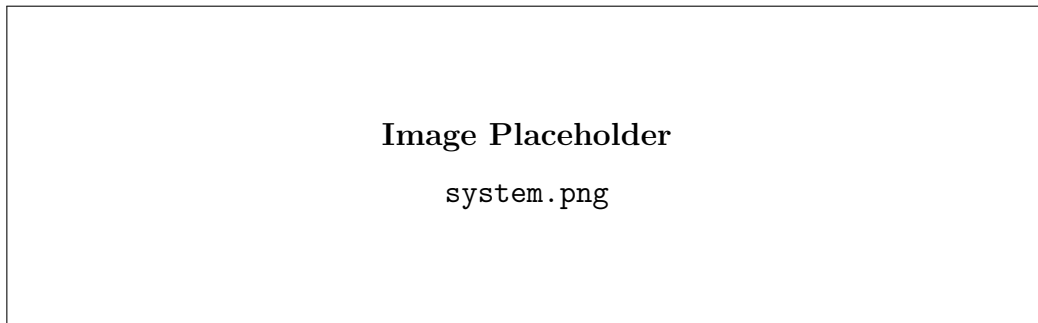


Figure 1: Game System

In the course “Principles of Artificial Intelligence,” we studied a variety of search, planning, and optimization algorithms. Based on these core methodologies, our Maze Escape project extends static, single-agent pathfinding into dynamic, multi-dimensional adversarial scenarios. To achieve this transition, we introduce randomly generated maze maps based on genetic algorithms, expand the dimensional space, and a two-way tracking mechanism enhanced by predictive machine learning has been realized.

In order to comprehensively evaluate these algorithms, we have designed two complementary game modes: escape mode and chase mode. In the chase mode, the actions of human characters are driven by evasion algorithms based on Breadth-First Search (BFS). For the core task of predators pursuing prey, the project adopts four distinct intelligent strategies and combined with machine learning-driven interception mechanism:

- **A* Search Algorithm:** In single-monster mode, the algorithm calculates the shortest path to the player. In dual-monster mode, monsters are divided into two types: direct pursuers and interceptors. The interceptors predict the player’s next move and block potential escape routes.
- **Greedy Algorithm:** It calculates a distance field centered at the player’s current position, causing monsters to always move toward the nearest neighboring node. This pursuit method is highly efficient and requires low computational resources.
- **Minimax Decision Tree:** This strategy treats the pursuit process as a two-player zero-sum game. The algorithm combines α - β pruning with a transposition table to predict subsequent moves. The monster aims to minimize the player’s safety score during the game, while simultaneously simulating the player’s efforts to maximize it, thereby enabling efficient pursuit from both perspectives.
- **Simulated Annealing Algorithm:** The algorithm first generates an initial movement path and then randomly adjusts it through repeated iterations. Following the cooling schedule, the system accepts suboptimal moves with a certain probability, making the monster’s actions more stochastic and harder to predict.
- **Machine Learning:** Based on spatial characteristics (the relative position of humans relative to monsters, local wall obstacles and the direction of movement of the first two steps), we have trained a random forest model to predict the next move of human players, and predict the position of the next three steps by rolling, so as to achieve effective blockade.

The environment of our game is a 30×30 circular grid map, in which the specific layout of the maze is generated by the adjusted genetic algorithm (GA). The algorithm ensures that the highest quality map is finally obtained by maintaining a candidate map population and evolving it for more than 20 generations. The adaptability function of the genetic algorithm comprehensively considers several key factors, including good ground connectivity, about 28% of the number of obstacles, the distribution of three-way and four-way intersections, and the breadth-first search (BFS) path length with an average of nearly 20 steps between randomly selected positions. We also adopt methods such as tournament selection, row crossing and bit flip transformation to effectively help the algorithm generate maps with high accessibility, spatial diversity and appropriate difficulty.

The following chapters of this report will introduce in detail the design and specific implementation methods of each component, quantitatively analyze the algorithmic characteristics of each artificial intelligence strategy and its impact on gameplay, and reflect on their respective advantages and limitations observed during the development process.

2 Methodology

2.1 Genetic Algorithm for Map Generation

In this project, we automatically generate the map through a genetic algorithm rather than designing it manually. The purpose of this module is not simply to place obstacles randomly, but to generate a 30×30 map that is suitable for a chase-based game.

Therefore, we need the map to satisfy connectivity, playability, and chase balance at the same time. If the map is too open, the monster can chase the player too easily. If there are too many obstacles, the movement space of both the player and the monster will be severely restricted. In some extreme cases, the player and the monster may even be separated into unreachable areas. Therefore, we design the map generation process to balance randomness and playability.

In the implementation, we model the map as a 30×30 two-dimensional grid, and every cell is represented using binary values. 0 symbolizes walkable ground, while 1 symbolizes obstacles and walls. Players and monsters can only move on walkable cells. Besides, we design the map with a wrap-around boundary. This means that when a character moves out from one side of the map, it will appear again from the opposite side. Because of this mechanism, modulus operation is used when calculating neighbouring positions. BFS can also be applied based on this wrap-around structure to check map connectivity and calculate the path distance between the player and the monster.

In the genetic algorithm, we regard each map as a chromosome, while a group of maps forms a population. The initial population is generated randomly, where each cell is assigned as either walkable ground or obstacle according to a fixed wall probability. In this implementation, we set the target wall ratio to around 0.28, which means that about 28% of the map is expected to be walls. This value is used to avoid maps that are either too empty or too crowded. After the initial maps are generated, each candidate map is evaluated by a fitness function. The fitness function mainly considers four factors: reachability, wall ratio, junction complexity, and path distance. The final fitness score can be expressed as:

$$\text{Fitness} = S_{\text{reachability}} + S_{\text{wall ratio}} + S_{\text{junction}} + S_{\text{path distance}} - P_{\text{dead-end}} \quad (1)$$

Reachability is used to check whether most walkable cells are connected. Wall ratio controls the proportion of walls and compares it with the target value of 0.28. Junction complexity measures the number of branches and intersections, which can provide more possible route choices during the chase. Path distance is used to evaluate whether the average BFS path length in the map is suitable for the survival chase setting. The dead-end penalty is added to reduce the number of cells with very few exits, because too many dead ends may make the map unfair or less playable.

During each generation, the generator calculates the fitness score of every map in the current population. Then, maps with better scores are selected as parents through tournament selection. The crossover operation combines partial structures from two parent maps to generate offspring maps. The mutation operation randomly changes a small number of cells with a low probability, such as changing walkable ground into walls or changing walls into walkable ground. After several generations of selection, crossover, and mutation, the system gradually evolves maps that are more suitable for the game environment.

At the end, the map with the highest fitness score is saved as a text file. The map file uses a space-separated 0 and 1 format, where each line corresponds to one row of the map. This format is simple, readable, and easy for both Python and C++ game modules to load. Apart from the map grid itself, the system also validates the spawn points of the human and monsters. The spawn points must be located on walkable cells, and the human and monster spawn points cannot overlap. The system also checks the BFS path distance between them. In this implementation, the valid starting distance is set between 10 and 25 BFS steps, so that the monster is not placed too close or too far away from

the human at the beginning of the game.

2.2 Human AI Algorithm (BFS)

When the user plays the role of monster, a Breadth-First Search(BFS)-based system is used to control the human character. Generally, the objective of the human character is to keep as far away from the monster as possible. Therefore, every time when the Human AI needs to decide its move direction, we run BFS starting from the monster's current position. This creates a distance map over the whole maze, where each reachable cell stores how many steps the monster would need to reach it. Since the maze is an unweighted grid, BFS gives the shortest path distance correctly.

After building this distance map, the Human AI checks the four possible next positions of the human: up, down, left, and right. It chooses the move with the largest BFS distance from the monster, meaning the human tries to move to the neighboring cell that is farthest away in terms of actual maze path distance.

2.3 Monster Algorithm

2.3.1 A* Search Algorithm

A* is a heuristic graph search algorithm. It finds the shortest path between two nodes by adding the cost of the lined path to the residual cost estimate of reaching the target node. The cost of each node is calculated by the following function:

$$f(n) = g(n) + h(n)$$

In the process of the monster moving to the player in each round, the map grid is regarded as an untitled map, thus calculating the path cost. Due to randomly generated roadblocks, each walkable grid cell has up to four neighbors, which can be reached by moving up, down, left or right. When calculating heuristic functions, we use the Manhattan distance between the current cell and the player's location. In addition, because the map has a ring structure, we will apply modular operations to the row index and column index every time we expand the neighbor. This allows movement to cross the grid boundary. After the algorithm is executed, the core function `astar(grid, start, goal)` will return the full path from the current position of the monster to the player. With each move of the player, the Astar algorithm will calculate the corresponding optimal path. If there is no path (for example, the player is completely surrounded by a wall), the monster will stay in place.

2.3.2 Greedy Search for Monster Movement

The Greedy Search is our basic pursuit strategy, which applied in the "simple" level. At each beginning of each round, our system will run a BFS computation at the current cell, in order to build a distance field: the array stores the real distance between player and monster (with grid steps, including the circumstance of walls and wrap-around) to human. After that, each monster check all the cell it can move to, and choose the shortest one, and repeat it again. The system avoid two monster stayed in the same cell. If the monster is stuck, and have no possible way to player, we will use Manhattan distance, so the monster still makes forward progress.

The Greedy Search is very speedy and simple, each round and only need to compute BFS once and a constant number of comparison. Its shortcoming is that the monster is totally passive during the game, can not predict where the player would go. Facing the clever player, it would be dumb and be stuck in the trap, so it will be more long for it to catch the player. Hence, the Greedy Search is a well-known basic strategy, the foundation of the higher or cleverer algorithm.

2.3.3 Simulated Annealing Algorithm

At the third difficulty level, Simulated Annealing (SA) is used to plan the monster's movement. Unlike A* or BFS, SA does not require every intermediate candidate to be a wall-free shortest path. Instead, it searches over complete, continuous routes from the monster to the human and assigns large costs to routes that cross walls. This representation makes mutations inexpensive while allowing the algorithm to explore routes that differ substantially from the current one. Since the monster executes only the first two steps of a route before replanning, the objective gives particular importance to the beginning of the path and a wall-aware BFS is retained as a final safety mechanism.

The maze is treated as a torus. For two cells $a = (a_x, a_y)$ and $b = (b_x, b_y)$, their heuristic distance is

$$Dist(a, b) = \min(|a_x - b_x|, 30 - |a_x - b_x|) + \min(|a_y - b_y|, 30 - |a_y - b_y|).$$

The initial state is a random toroidal Manhattan shortest path from the monster to the human. The required horizontal and vertical moves are calculated first and then shuffled. When two wrap-around directions have the same length, one is selected randomly. Consequently, the initial path is continuous and always ends at the human, but it deliberately ignores walls. Wall avoidance is then introduced through mutation and the energy function.

To generate a neighbouring state, the algorithm selects two points on the current path and rebuilds the segment between them. With probability 0.70, the left endpoint is fixed at the monster's current position so that the mutation can change the two steps that will actually be executed. Half of these early mutations rebuild the path all the way to the human; the other half use a right endpoint no more than 30 path positions away. For a non-early mutation, the left endpoint is sampled uniformly from the path and the right endpoint is selected within the same 30-position span.

The replacement segment is constructed by a wall-aware greedy search for at most 80 steps. At each step, it chooses a walkable neighbour with minimum toroidal Manhattan distance to the segment endpoint. The algorithm prefers unvisited cells and moves that do not immediately backtrack. Meanwhile, candidates with the same expected distance are shuffled to preserve variation. If the greedy search does not reach the right endpoint within a certain number of steps, a random toroidal shortest path is appended to close the segment. This closing part may cross walls, but the resulting route must remain continuous, start at the monster, end at the human, and contain no more than 140 cells. A candidate that fails these structural checks is rejected immediately.

For a path $P = (p_0, p_1, \dots, p_{n-1})$, let h be the human position and $p_{\text{move}} = p_{\min(2, n-1)}$ be the position reached after the move executed in the current turn. The energy minimized by SA is

$$E(P) = |P| + 3Dist(p_{\text{move}}, h) + Wall(P) + R(P).$$

The path-size term favours short routes, and the larger weight on $Dist(p_{\text{move}}, h)$ encourages the monster to move towards the human to maintain pursuit pressure.

The wall cost distinguishes cells near the executable part of the route from those farther in the future:

$$Wall(P) = \sum_{i=1}^{n-1} \begin{cases} 10000, & p_i \text{ is a wall and } i \leq 6, \\ 100, & p_i \text{ is a wall and } i > 6, \\ 0, & p_i \text{ is walkable.} \end{cases}$$

The very large early penalty strongly penalizes an illegal current move. The smaller later penalty still guides the search towards a fully walkable path, while also permits SA to pass through temporarily poor intermediate states. Because a new route is planned every turn, the first six steps are attached more importance than a distant suffix that may not be executed instantly.

The final term reduces visible repetition between turns:

$$R(P) = 10I(p_{\text{move}} = p_{\text{prev,start}}),$$

where $p_{\text{prev,start}}$ denotes the monster position at the beginning of the previous round. The game backend records the previous movement and passes it to the C++ planner on the next turn. Thus, returning immediately to the previous position is discouraged without preventing the monster from continuing a useful direction of pursuit.

A candidate P' with improvement is always accepted. A worse candidate might also be accepted with probability

$$\Pr(P \rightarrow P') = \exp\left(\frac{E(P) - E(P')}{T}\right).$$

This permits broad exploration at high temperature and increasingly selective search as the temperature falls. The implementation also tracks the best accepted route whose first two steps are both walkable. That route is preferred over a lower-scoring route that the monster cannot execute safely.

Table 1: Default simulated annealing parameters

Parameter	Value
Candidate trials per temperature	20
Initial temperature	80
Minimum temperature	0.5
Cooling rate	0.94
Immediate-move distance weight	3
Normal wall penalty	100
Wall penalty in the first six steps	10000
Greedy segment-rebuild budget	80 steps
Maximum local segment span	30 path positions
Probability of an early mutation	0.70
Maximum candidate path size	140 cells

With these defaults, the temperatures are $80, 80(0.94), 80(0.94)^2, \dots$ while $T > 0.5$. This gives 83 temperature levels and $83 \times 20 = 1660$ candidate mutations per call.

After annealing, the monster chooses the best accepted route with legal first steps. If no such route is found, a wall-aware toroidal BFS constructs a safe fallback path. The selected route is checked again before output, ensuring that the monster never crosses the wall even though SA is allowed to evaluate wall-crossing candidates during the search.

The current configuration was chosen based on an ablation experiment. We tested the algorithm on five generated maps with five fixed random seeds per map, it caught the human in 24 of 25 games. Its mean planning time was 6.72 ms, the 95th percentile was 10.16 ms, and the maximum was 12.35 ms. No BFS fallback was required and the selected routes contained no wall visits in these runs. These results show that the method is fast enough for interactive play. Also, the algorithm can effectively retain safe movement while producing varied routes.

2.3.4 Minimax for Predictive Monster AI

Minimax models the chasing process of monster and player as a zero-sum game. The monster as the Min, wanting to shorten the distance between one another, while the player is the Max, wanting to make the distance larger. The system builds a depth-limit game tree which represent every step or action the agent might move. Following the round-round structure, at each round, the monster will search and take two step forward, then play will take one-step action. The round will be repeated with a constant number rounds, the default depth is two, so that it will be within the computation budget.

Leaf nodes are calculated by an evaluation functions, which is based on the distance of player to each monsters.

$$\text{Eval} = 4 \cdot \min_i d(\text{Human}, \text{Monster}_i) + \sum_i d(\text{Human}, \text{Monster}_i)$$

We give the first term as a approximately weight 4, which is the biggest weight as well, since the smallest distance represent the biggest risk. The summation keeps the other monsters moving as well. Small figure favor the monsters, while the big one favor the player. If the monster get the player's cell, the node will return a big negative score, and if the use step is the score is lower, so that the monster will try to use the least step to catch.

Searching the whole tree is expensive, so we use standard optimizations. First, we use the alpha-beta pruning, to eliminate the branch which is impossible to effect the final decision, so that we can cut down the quantity of the nodes to be searched. Second, we apply the transposition table to cache the result of each evaluated state. With player position, tuple of monsters position, and ply index as keys and all together with a bound flag. Since all in a wrap-around grid, so we can avoid a lot of repeated work and calculation. Third, we use the move ordering to check the most hopeful path first: the path of monster are ordered by increasing distance to player, the path of monster are ordered by decreasing distance to player, and the best path recorded by the transposition table. Good sequence will make the alpha-beta prune much more aggressively and get a better result. After the searching, the chosen sequence of move for current move will be rebuilt from the transposition-table entries and returned through the same interface.

Compared to Greedy Search, Minimax and do predict to the player, more predictable. Since it will modify simulate the best choice made by player, so it will do cut the escape route of the player, but not just follow the player passively. But the cost is high computation, and increasing with branching factor and look-ahead depth. The optimization

above is the key factor that make the algorithm into usable. Hence, Minimax will be used in the more difficult level.

2.4 Machine Learning Dual-Monster Mode

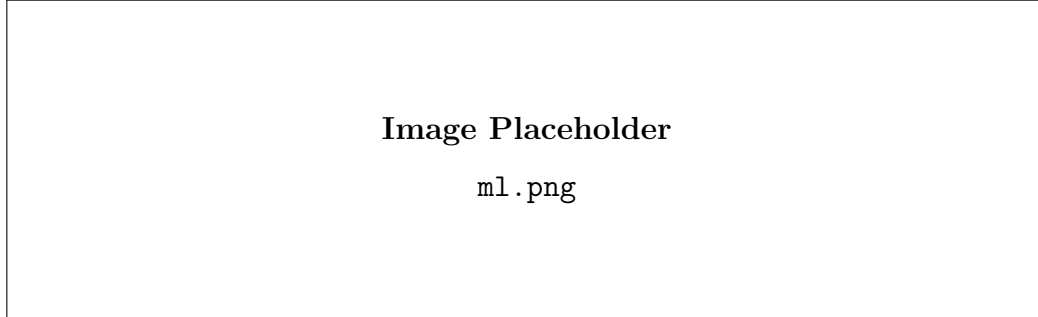


Figure 2: Our Work

Listing 1: Serialized JSON Structure of the Gameplay Instance

```
{
  "features": {
    "rel_row": -3,
    "rel_col": 5,
    "wall_up": 0,
    "wall_down": 1,
    "wall_left": 0,
    "wall_right": 0,
    "prev1": 2,
    "prev2": 1
  },
  "label": "left"
}
```

To achieve efficient collaborative pursuit in the dual-monster mode, this project introduces a machine learning approach to predict the human player's next movement direction. Given that the game map represents a non-linear discrete space, we selected a Random Forest classifier to construct the predictive model. The training data was collected entirely through real gameplay testing conducted by our group members, resulting in a total of 4,206 data entries saved in JSONL format. Each line represents a player decision sample containing the current state features and the ground-truth movement label. Based on this dataset, to minimize the parameter count while preserving high-quality characteristics, the model extracts and learns three core categories of features:

1. **Spatial Relative Position:** The two-dimensional coordinates of the human relative to the monster, using the monster as the reference frame.
2. **Environmental Obstacles:** Binarized features indicating whether the human is blocked by walls in the four adjacent directions.
3. **Historical Behavior:** The human player's movement directions during the previous two actions.

The output target of the model is the human’s movement direction in the next step.

During training, all record data is first loaded from the file `replay_log.json`, and then each record is converted into a corresponding feature matrix and label vector. Subsequently, the dataset is divided into an 80% training set and a 20% test set, and fed into a **random forest classifier** for training. Table 2 details the hyperparameter configuration of our **random forest model**. Experimental results show that the model exhibits excellent generalization ability on the test set, achieving an accuracy of 77.08% in predicting the next direction.

Table 2: Random Forest Hyperparameters

Hyperparameter	Value	Description
<code>n_estimators</code>	300	Number of decision trees
<code>max_depth</code>	12	Maximum depth of each tree
<code>min_samples_leaf</code>	3	Minimum samples at a leaf node
<code>class_weight</code>	balanced	Handles class imbalance
<code>random_state</code>	42	Ensures reproducibility

In the dual-monster mode, one monster uses the A* algorithm to directly track the player’s current position, while the other monster uses the player’s predicted position as the target point.

$$\begin{aligned}
 p_{t+1} &= p_t + \text{move}(\hat{d}_{t+1}) \\
 p_{t+2} &= p_{t+1} + \text{move}(\hat{d}_{t+2}) \\
 p_{t+3} &= p_{t+2} + \text{move}(\hat{d}_{t+3})
 \end{aligned}$$

where p_t denotes the player’s current coordinate, and $\hat{d}_{t+1}$, $\hat{d}_{t+2}$, and $\hat{d}_{t+3}$ represent the predicted movement directions for the next three consecutive steps, respectively. The final obtained coordinate p_{t+3} is designated as the target point for the second monster (the interceptor). Through practical gameplay testing, the average spatial prediction error for the future third-step position is only around 2 grid cells. This anticipation mechanism allows the interceptor to seal off the human player’s escape routes in advance, creating a highly effective collaborative pincer attack alongside the primary monster that directly pursues using the A* algorithm.

3 Game System Design and Gameplay Integration

3.1 Overall Game Flow

3.1.1 Game Execution Process

The complete game flow of *Maze Escape* can be divided into three stages: initialization, the turn loop, and end-game determination and reset. These form a closed loop that runs from start to finish, then resuming with the next game.

1. Initialization Stage

Once a player selects “START GAME”, the first screen is to choose which character to control for that round. If they choose “PLAY AS HUMAN”, they must first choose a difficulty level before entering the game. If selecting “PLAY AS MONSTER”, they enter the game immediately. Before the game begins, the system uses a genetic algorithm to generate the map. It will determine the initial item spawn points, and set the starting coordinates for both the human and the monster. It will also ensure there is a minimum safe distance between them to prevent the human from being captured right at the start.

2. The turn loop

In each turn, the player-controlled character always moves first, then is AI-controlled character’s turn. The human character moves one square per turn, and the monster moves two. Each time the coordinates are updated, the number of moves is refreshed to the HUD, and the system immediately checks whether the two characters’ coordinates are overlap. this check serves as the link between the turn cycle and the end-of-game determination. In human mode, the system also checks whether the player has stepped on an item, in order to trigger the item system’s mechanics.

3. End-Game Evaluation and Reset

Once the coordinates of the human and the monster overlap, the game ends and the system transition to the End screen. On the End screen, players can choose “PLAY AGAIN”, which refreshes the map but retains the original mode and difficulty settings. Or player can select “MAIN MENU”. Once the reset is complete, the process returns to stage 1, forming a complete loop.

3.1.2 Interface Navigation and Global Button Rules

From the frontend perspective, the above process is reflected in the actual page transitions as follows: Index → Character select → (human via Difficulty) → Game → End. The flowchart is shown below:

3.2 Game State Representation

The game state is consolidated a single `GameState` data class. each game corresponds to a separate instance, which is stored in the global dictionary `games` using the key `game_id`, thus supporting the concurrent running of multiple games.

1. Grid and Coordinate System

The map is represented as a two-dimensional matrix. Each cell in the matrix designated solely as either passable space or impassable walls. Furthermore, the map boundaries are designed as a loop, allowing characters to re-enter from the opposite side when they cross the map boundary. The positions of players, monsters and items are each recorded as separate data points.

2. Entity States

A player’s state mainly records their current position and the short-term effects resulting from picking up items. A monster’s state records its current position and whether it is under the influence of any items.

3. Mode and Algorithm Binding

Algorithms for different modes and difficulty levels are contained within separate planning stages. The mode will display on the HUD at the start of the game. This binding ensures that the state structure itself remains independent of the algorithms.

4. Item and Effect States

Before picking up items, they are placed on the grid. Once human going to the item grid, they become a temporary status effect on the player or monster. In this game, items take effect immediately and gradually fade as the turn progresses.

3.3 Turn-Based Movement Logic

Based on the set turn described in 3.1, this section further explains the specific logic for judging during movement.

By default, players move one square per turn, while monsters move two squares. Before each move, the system first checks whether the target square is accessible. If it is not, the move is immediately rejected and the player's status will not change. Both collision and capture evaluations are performed after every single-square move, rather than just once at the end of the turn.

As the number of moves a player can make each turn is affected by items, we do not assume a standard number of moves per turn. Instead, we check whether the player has any moves left in the current turn. It is only when the player has used all their moves for that turn, or when a particular effect causes the movement phase to end prematurely, that it becomes the monster's turn to act.

The monster's movement of two squares is processed step by step, rather than as a single jump. After each square is covered, the system checks whether the monster catches the human. And the second step is not performed if the first capture. This approach also allows the frontend to display the monster's intermediate positions, resulting in a smooth movement animation.

3.4 Interactive Item System

The item system within this project was designed as a significant component of the game logic, rather than merely a decoration on the map. Since the monsters can chase the player using algorithms such as A* Search, Greedy Search, and machine-learning-based prediction models, the game would become too difficult if the player could only move one step per turn. Therefore, the main purpose of the item system is to provide short-term escaping, defensive, and repositioning abilities, which improves the playability and balance of the game.

This system includes four items: **Swift Boots**, **Frost Trap**, **Teleport Stone**, and **Cloak**. These four items affect the player's movement ability, the monsters' movement status, the player's location, and the monster-chasing logic respectively. All item effects are integrated with the turn-based movement system. In most cases, the duration of an item effect is updated based on a complete player turn, rather than based on a single movement cell.

3.4.1 Swift Boots

Swift Boots is a movement-enhancing item. After the player picks up this item, the player can perform at most two separate one-cell movements in each of the following five complete player turns. These two movement steps can be in different directions. For example, the player can first move upward and then move to the right. Therefore, this item does not simply allow the player to move two cells in a straight line. Instead, it changes the player's turn structure and allows the player to perform multiple movements within one turn, similar to the movement pattern of the monsters.

This design is more flexible than directly moving two cells in one direction, because the player can turn corners to avoid walls or escape from the monster's chasing path. Each single-cell movement is still checked independently for wall collision, item pickup, and monster collision. As a result, the player cannot pass through walls or skip important events on the map.

The duration of Swift Boots decreases by one only after the player finishes a complete turn, instead of decreasing after each single movement. If the player picks up another Swift Boots while the effect is still active, the duration will be refreshed to five turns instead of being stacked repeatedly. This mechanism ensures that the item is useful enough for escaping, while preventing the player from staying in an overly powerful state for too long.

3.4.2 Frost Trap

At first, we only tested the normal chasing logic. However, in the two-monster mode, the player could be easily caught by two monsters. Therefore, we decided to design an item called Frost Trap. In the implementation, we did not simply pause the whole game. Instead, we saved a frozen counter for every monster individually. After the player picks up this item, every monster's counter is set to 5. Every time a monster prepares to move, the program first checks this value. If the value is larger than zero, the monster cannot move in this turn. After the player finishes a complete turn, the counter decreases by one.

We also considered whether freezing only one monster would be enough, but the effect was not good in the dual-monster mode, because the other monster could still move close to the player. Therefore, we finally made Frost Trap affect all monsters at the same time. To avoid making this item too strong, when the player picks up this item several times, the frozen time will not be accumulated without limit. Instead, we refresh the counter or keep the larger frozen value.

3.4.3 Teleport Stone

At first, Teleport Stone could be designed as a fully random teleportation item, but we thought this was not stable. This is because the random position could still be close to the monsters, or even teleport the player to a more dangerous position. Therefore, we did not simply choose a grid randomly. Instead, the program goes through all the grids on the map. The program first excludes positions that cannot be used as teleport targets, such as walls, the current position of the player, monster positions, active item positions, and trap positions. The remaining grids are then added to the candidate list. After that, we calculate the distance from each candidate grid to the nearest monster. This distance is used as the safety score of the position. If the distance is larger, it means that the

position is safer. Finally, we do not directly choose the farthest grid, because the result would be too fixed. Instead, we randomly choose one position from the top 10% safest positions. In this way, the player has a high probability of being teleported to a safe area, but the result still keeps some randomness.

3.4.4 Cloak

Cloak is a stealth-type item. When the player picks up this item, the player becomes invisible for ten complete player turns. During invisibility, monsters cannot detect or capture the player. Instead of using A* Search, Greedy Search, or machine-learning-based prediction to chase the player, the monsters move randomly among valid neighbouring cells.

This design changes the behaviour of the monsters during the invisible state. The monsters are still active in the maze, but they no longer have a clear target. Therefore, the game does not become completely static when the player is invisible. At the same time, even if a monster randomly moves onto the player's position, the game-over condition will not be triggered while the invisibility effect is active.

The duration of Cloak is also updated based on complete player turns. This is important because Swift Boots may allow the player to perform two movement steps within one turn. In this case, the Cloak duration should still decrease by only one after the whole player turn is completed, rather than decreasing twice. This keeps the time-based logic of all item effects consistent.

After the invisibility duration ends, the monsters return to their normal chasing behaviour and collision detection becomes active again. Since Cloak can completely prevent the player from being captured in a short time, it is designed as a rare and powerful item.

3.4.5 Item-State Update and Game Balance

In our game, we count item effects by player turns rather than every single movement. This is important because Swift Boots can let the player move twice in one turn. If the effect duration were reduced after each movement step, Swift Boots and Cloak would lose time too quickly and the item logic would become inconsistent. Therefore, we update the remaining turns of Swift Boots, Frost Trap, and Cloak only after the player finishes the whole turn.

The item system also makes the game fairer for the human player. The monsters can chase the player with search algorithms and prediction models, so we give the player some short-term abilities to survive. Swift Boots gives extra movement, Frost Trap stops the monsters for a short time, Teleport Stone moves the player to a safer position, and Cloak prevents the monsters from catching the player temporarily. These items do not remove the challenge of the game, but they give the player more choices when the monster AI becomes too strong.

Overall, we use the interactive item system to increase the complexity and playability of the game. Swift Boots improves the player's mobility, Frost Trap provides temporary control over the monsters, Teleport Stone allows emergency repositioning, and Cloak temporarily prevents the monsters from targeting the player. These items work together with the turn-based game logic and make the maze escape process more strategic and dynamic.

Table 3: Summary of Interactive Item Effects

Item	Type	Effect
Swift Boots	Mobility	Allows the player to perform two separate one-cell movements per turn for five complete player turns. The two movement steps can be in different directions.
Frost Trap	Control	Immediately freezes all monsters for five complete player turns. Frozen monsters cannot move or continue chasing the player.
Teleport Stone	Escape	Teleports the player to a randomly selected cell from the top 10% safest valid cells, based on the distance to the nearest monster.
Cloak	Stealth	Makes the player invisible for ten complete player turns. During invisibility, monsters move randomly and cannot capture the player.

3.5 User Interface and Backend Integration

The entire game’s user interface employs a consistent pixel-art forest theme, modelled on the style of *Stardew Valley*. In addition, the front-end interface maintains tight data coordination with the back-end system:

When every action taken by the player, it triggers a request to the Flask back-end. After updating the game state, the back-end returns structured data, and the front-end renders the interface based on this.

The following sections will introduce several core interface components, detailing how front-end and back-end synchronisation is implemented. The following sections introduce the main interface components and explain how front-end and back-end are coordinated

3.5.1 Pre-Game Pages

The Index page serves as the main entry point of the game. It employs a wooden board-style title design and provides three main options: “Start Game”, “Manual”, and “Exit”. The manual interface provides a structured overview of the introduction of two modes, an explanation of the control scheme and four items. To enable players to review the rules at any time during the game, pressing the M key on any screen can open a overlay component of the manual. It allow them to quickly check methods and item effects without having to exit the game.

The Character Selection screen allows players to choose between ‘Monster’ and ‘Human’ modes. Each option is displayed as a pixel-art card, which is highlighted when the mouse hover is over it; clicking on it plays a short loading animation. If the player selects Human mode, they will be taken to the Difficulty Selection screen, where different difficulty levels are distinguished by different colors. After clicking, the system displays a confirmation prompt, adding a layer of error tolerance.

Although the front-end interaction pages are primarily static interfaces, they all rely

on data exchanges with the back-end API to initiate game session. Once the player completed their selections on these two screens, the front-end sends the chosen game mode and algorithm(`opponent_ai`), along with other settings, to the API via a POST request.

Upon receiving a request, the backend calls a function to generate the map and initially sets up the monster path-planning logic, the initial positions of each character, and the distribution of items. Once they are complete, the backend returns the complete game state to the frontend in JSON format via the `GameState.serialise()`, and the frontend renders the interface according to this. It is evident that the connection between the front-end interaction page and the backend ensures that game environments across different modes and difficulty levels can be correctly created and distinct.

3.5.2 Gameplay Interface

The Game page maintains the overall pixel style in its visual presentation. The map area on the left is drawn using Canvas, and the static wall sections are optimised with offscreen canvases to reduce the repeat rendering. The player’s movement animations employ smooth transitions based on linear interpolation (`lerp`), and the `inputLocked` state is used to lock input during gameplay, preventing new inputs to interrupt the current animation.

The HUD panel on the right consists of two sections: “Game Stats” and “Item”. The Game Stats section displays the current difficulty level, the number of steps taken by both humans and monsters, and the number of times items have been used. The Item section displays the four core items, with corresponding status text and a countdown timer showing when the items will next be refreshed displayed below each icon. These values are updated in real time based on status data returned from the back-end.

In Human Mode, every movement command is sent from the frontend to the backend.

It enable the `apply_player_move()` function to process the logic for that turn. First, it checks whether the target square is accessible. If the move is viable, it then checks whether there are any items at that location and activates the relevant effect. Due to the existence of the *Swift Boots*, the number of moves within a single turn is not limited to one. We use status variables like `remaining_player_steps_this_turn` and `waiting_for_second_step` to dynamically determine whether the player is still able to make a second move. And based on this, the system can decide whether to end the turn early or let player make an additional move.

When this round’s movement is finished, the backend calculates the monsters’ new positions using the algorithms we prepared. Then it updates status decay information such as item duration and the number of freeze rounds. Finally, it returns the complete state to the frontend via `serialise()`. So the map and HUD can be refreshed automatically.

In Monster Mode, we use a simpler interaction process. This mode does not include the item system. When the player makes a move, the system first checks whether the target position is valid. If the move is allowed, it will updates the monster’s position. Then the human moves based on the BFS algorithm after every two player moves. We repeat this process until player catches the human. Compared with Human Mode, we handle fewer interactions and use less complex game logic.

In addition, we designed a wall collision warning pop-up window to demonstrate how the front-end and back-end work together to handle exceptional circumstances. When a player attempts to move, the back-end checks the passability of the target square using

`move_one()` and `move_human_with_items()`. If it judges the square to be a wall, it returns the respective error message. Once the front-end receives this response, it triggers a pop-up alert. It presents the back-end's result as clear visual feedback and preventing players' confusion caused by a lack of immediate feedback.

3.6 Deployment Architecture

To allow users to play the game without setting up a local environment, we deploy the Flask backend on the *Railway* cloud platform.

On the back end, we use *Gunicorn* as the WSGI server in the production environment, replacing the native Flask server used during the local development phase. During deployment, we limit the number of workers to 1 to ensure that all requests access the same game state. Since some of the algorithms in the project are written in C++, we created a custom *Dockerfile* to ensure that the deployment environment (Linux containers) has the ability to compile C++. The g++ compiler will be installed during the container build process, enabling the C++ modules to compile and run correctly in the cloud.

The code is managed on *GitHub*. Every time changes are pushed to *GitHub*, the *Railway* platform automatically detects them and rebuilds and redeploys the project.

4 Validation and Evaluation

4.1 Map Generation and Spawn Test

We tried to see if the generated map could be used within the game for the map generation part. The map could appear correct in the map file but might not be appropriate for the game. It must also allow normal movement, monster chasing, and valid spawn locations. Hence, before the map was loaded into the main game system, we performed checks on the map format, connectivity, wall density, and spawn distance.

The first check is the map format. Our grid has a fixed size of 30×30 , so the output file needs to have 30 rows and 30 columns. Also, every cell has to be either 0 or 1. In our map file, 0 represents a walkable tile, while 1 represents a wall. This check is required because the backend movement logic relies on this fixed format, and so does the canvas rendering.

The second check is about the connectivity within the map. The map can have the proper dimensions, but the walkable area can still be divided into several regions. In this case, the human and monsters might not be able to reach each other. We solved this problem by performing BFS to verify whether the walkable cells were well connected. Since our game uses wrap-around movement, we had to make the BFS check use the same neighbour rule as the actual movement logic.

The wall density was also checked. Here, we took the target wall ratio to be approximately 0.28. This value provides a suitable amount of obstacles in the map without making the maze too blocked. Too few walls allow the monster to pursue the human too directly, while too many walls make the movement space too constricted. Therefore, the wall ratio is added as a constraint in the fitness function to regulate the difficulty of the generated map.

We manually inspected the map structure and then tested the spawn locations. The human and monsters should begin on cells that can be walked on, and they should not be in the same cell. We used BFS path distance to measure the distance between the

human and the monster, rather than using Manhattan distance. This is because actual paths will be influenced by walls and wrap-around movement. In the final version, we set the starting distance between 10 and 25 BFS steps. This helps prevent the monster from catching the human immediately, while still keeping the chase active from the start.

Finally, the map that we created was saved as `map/generated_map.txt` and loaded into the main program of the game. Once loaded, we tried to see whether the human was able to move normally, whether the monsters were able to chase through the maze, and whether collision detection was working properly. If these functions work as they are supposed to on this map, then it can be assumed that the map generation module has been integrated into the game system. With these checks, the output map is not just a random grid. It turns into a playable game maze where spawning, movement, chasing, and game-state updates can be performed.

4.2 Algorithm Evaluation

In order to evaluate the performance of different algorithms from multiple angles, we conducted a series of experiments on four algorithms with the maze escape game as the platform, and took the algorithm type as the only independent variable to compare the performance of each algorithm. For each algorithm, we have run 30 games under the same game configuration, including the same map generation process, grid size, movement rules, birth point distance logic and maximum step limit. This can ensure that any performance difference is attributed to algorithm-driven decision-making to the greatest extent. Next, we will introduce the evaluation experiment and its results in detail.

This evaluation refers to three indicators: capture rate, number of capture steps (the number of steps required to capture players) and average decision-making time. First of all, we use the capture rate to measure the stability of this algorithm: for the number of capture steps, monsters driven by stronger algorithms should be able to successfully capture players with fewer step limits. Secondly, the fewer the number of capture steps, the better, but a balance should be achieved between chasing efficiency. Therefore, we finally calculated the decision-making time to compare the calculation overhead required by different algorithms for each decision-making.

- **Catch Rate:** According to the experimental results (as shown in Figure 4), A*, Greedy, and Minimax all achieved a 100% catch rate, demonstrating the high reliability of these three algorithms in a maze environment like maze-escape. However, the catch rate of Simulated Annealing (SA) was only 83.3%. Tracing this back to the underlying principles of the algorithm, we found that because SA is based on probabilistic optimization, it accepts certain sub-optimal moves during the decision-making process to explore other potential paths. While this mechanism offers advantages in certain optimization problems, it is detrimental in real-time pursuit scenarios. Accepting poor solutions introduces randomness into the monster's behavior, preventing it from catching the player within short-duration chases. This also indicates that deterministic search-based algorithms are better suited for real-time maze escape games than simulated annealing.
- **Capture Steps:** Among the four algorithms, Minimax achieved the best results, requiring an average of only 20.5 steps to capture the player (the specific distribution is shown in Figure 3). Greedy followed closely behind with an average of 20.8 steps, while A* required an average of 21.3 steps. The differences among these



Figure 3: Steps Distribution

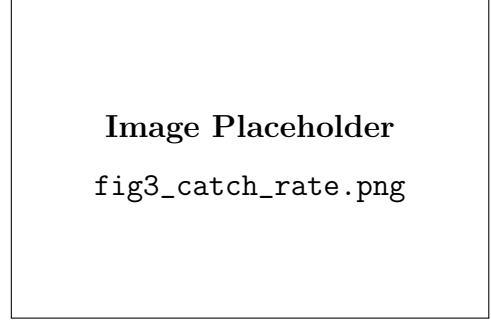


Figure 4: Catch Rate



Figure 5: Performance Overview

three algorithms are minimal, and they all share a median of 20 steps. This demonstrates that A*, Greedy, and Minimax are all capable of generating highly efficient pursuit behaviors in the maze. Based on the principles of the Minimax algorithm, we analyzed why it slightly outperforms the others: Minimax not only considers the player's current position but also uses lookahead search to preemptively restrict the player's future escape options. SA showed the weakest step performance. In successfully captured games, its average capture steps reached 26.8, which is significantly higher than the other three algorithms for the reasons mentioned above.

- **Decision Time:**

Table 4: Performance Comparison of Different Algorithms

Algo	Caught	AvgTime (ms)	MedTime (ms)
A*	30/30	0.0511	0.0342
Greedy	30/30	0.3449	0.3184
Minimax	30/30	0.6211	0.6299
SA	25/30	9.9962	9.8462

Considering that our game is a real-time interactive system, the AI must not only possess strong pursuit capabilities but also guarantee sufficient response speed. A* achieved the fastest decision speed, requiring an average of only 0.0511 milliseconds per decision. From an operational efficiency standpoint, it is the most outstanding among the four algorithms. Greedy's average decision time was 0.3449 milliseconds; although slightly slower than A*, it remains highly efficient. Minimax recorded an average decision time of 0.6211 milliseconds. Compared to A* and Greedy, its

execution speed is slower, but the reason is evident: Minimax needs to search future possible game states before making a decision. SA performed poorly, with an average decision time as high as 9.9962 milliseconds, far exceeding the other three algorithms.

- **Comprehensive Performance Analysis:** We aim to find the optimal trade-off between capture steps (efficiency) and decision time (speed), where the ideal algorithm should sit as close as possible to the bottom-left region of the chart 5. The chart shows that the A* algorithm delivers the best comprehensive performance, maintaining a low capture step count while delivering ultra-fast response times. Consequently, we deployed the A* algorithm in the game’s hardest levels, and the subsequent machine learning dual-monster mode was also developed based on A*. Minimax and Greedy represent two alternative solutions that achieve a good balance between capture steps and running speed. In contrast, Simulated Annealing performed the worst, lagging significantly behind the others in both aspects.

In conclusion, due to its exceptional execution speed, A* is the most outstanding all-round algorithm; however, when minimizing capture steps is the primary goal, Minimax performs the best. Conversely, the SA algorithm is entirely unsuitable for application in real-time pursuit games.

4.3 Machine Learning Evaluation

To evaluate whether the random forest model truly learned the characteristics of human movement behavior, we analyzed the model from two dimensions: feature importance and ablation study. Through these two analyses, we not only measured prediction accuracy but also understood which type of information contributed the most to predicting player behavior. Regarding the model’s prediction performance, the confusion matrix heatmap provides a comprehensive classification view. The model performs best when predicting "left" with a recall of 0.81, followed by "down" at 0.79, "right" at 0.75, and "up" at 0.71. This demonstrates that the model correctly predicts the vast majority of player movements across all four directions without any extreme bias.

To evaluate the performance of different algorithms from multiple perspectives, we conducted a series of experiments on four algorithms using a maze escape game as the platform, comparing their performance with the algorithm type as the sole independent variable. For each algorithm, we ran 30 games under the same game configuration, including the same map generation process, grid size, movement rules, spawn point distance logic, and maximum move limit. This ensures that any performance differences are attributable to algorithm-driven decision-making. Below, we will detail the evaluation experiments and their results.

This evaluation considered three metrics: capture rate, capture steps (the number of steps required to capture the player), and average decision time. First, we used the capture rate to measure the algorithm’s stability: for capture steps, a more powerful algorithm-driven monster should be able to successfully capture the player within a fewer step limit. Second, fewer capture steps are not necessarily better; a balance must be struck between capture efficiency and pursuit efficiency. Therefore, we finally calculated the decision time to compare the computational cost required for each decision by different algorithms.

First of all, according to the analysis of the importance of the characteristics of the random forest model, (Figure 6b) the model does not only rely on a single variable; the current spatial location, obstacle information and historical movement information are equally important, among which historical movement information is the most critical feature, and the importance score of `prev1` (the previous move) is 0.2842, followed by `prev2` (move the first two steps), the important score is 0.1463. It is reflected that in the actual game scenario, players usually continue to move in the same direction when escaping. In addition, spatial characteristics are also very important: the importance of `rel_col` (relative column distance) is 0.1443, and the importance of `rel_row` (relative row distance) is 0.1211. These two characteristics describe the position of the player relative to the monster, so that the model can learn what the player tends to reflect when the player and the monster are in different relative positions. The importance of wall features is low, between 0.07 and 0.08, indicating that obstacle features are mainly used to help the model exclude impossible movement.

The SHAP swarm diagram (Figure 6c) further explains how each individual feature affects the prediction results of the model. The content reflected by the swarm diagram is basically consistent with the previous feature importance analysis. The SHAP value of `prev1` is the most widely distributed, indicating that it has the greatest impact on prediction, while the narrower SHAP distribution of the relative position indicates that they need to work with historical information and obstacle information to play a role, and can rarely dominate the prediction alone.

The ablation experiment (Figure 6d) confirms the importance of reasonably combining all characteristic groups. The complete model achieves the highest balance accuracy rate of 0.7649. After removing historical features, the prediction accuracy will drop to 0.5712, which shows that the previous direction of movement is indispensable. However, the model that only retains historical features (`history_only`) has reached 0.6143, which is better than the position-only model and the wall-only model.

5 Conclusion

5.1 Results

We design and implement a web-based game system managed by a Flask backend. The backend employs an in-memory session architecture and uses `game_id` tracking to independently manage the simultaneous states of multiple clients. The system supports two game modes: “Escape Mode”, where players evade AI agents, and “Chase Mode”, where players pursue agents guided by breadth-first search (BFS).

For map generation, we implemented a genetic algorithm (GA) in C++ to procedurally generate 30×30 circular mazes. The algorithm constructs and optimizes a fitness function that combines reachability, wall-to-floor ratio, intersection complexity, and path distance, thereby generating non-deterministic maze topologies.

The system integrates a variety of wayfinding algorithms for the intelligent agents. Specifically, it includes the A* algorithm for shortest-path tracking, the Greedy algorithm for low-cost heuristic chasing, the Minimax algorithm for adversarial planning combined with Alpha-Beta pruning, and the simulated annealing (SA) algorithm for probabilistic movement. Frontend state rendering is implemented using HTML5 Canvas, while communication with the backend REST API is handled through asynchronous requests. Moreover, the backend integrates a cross-platform compilation pipeline to support dy-

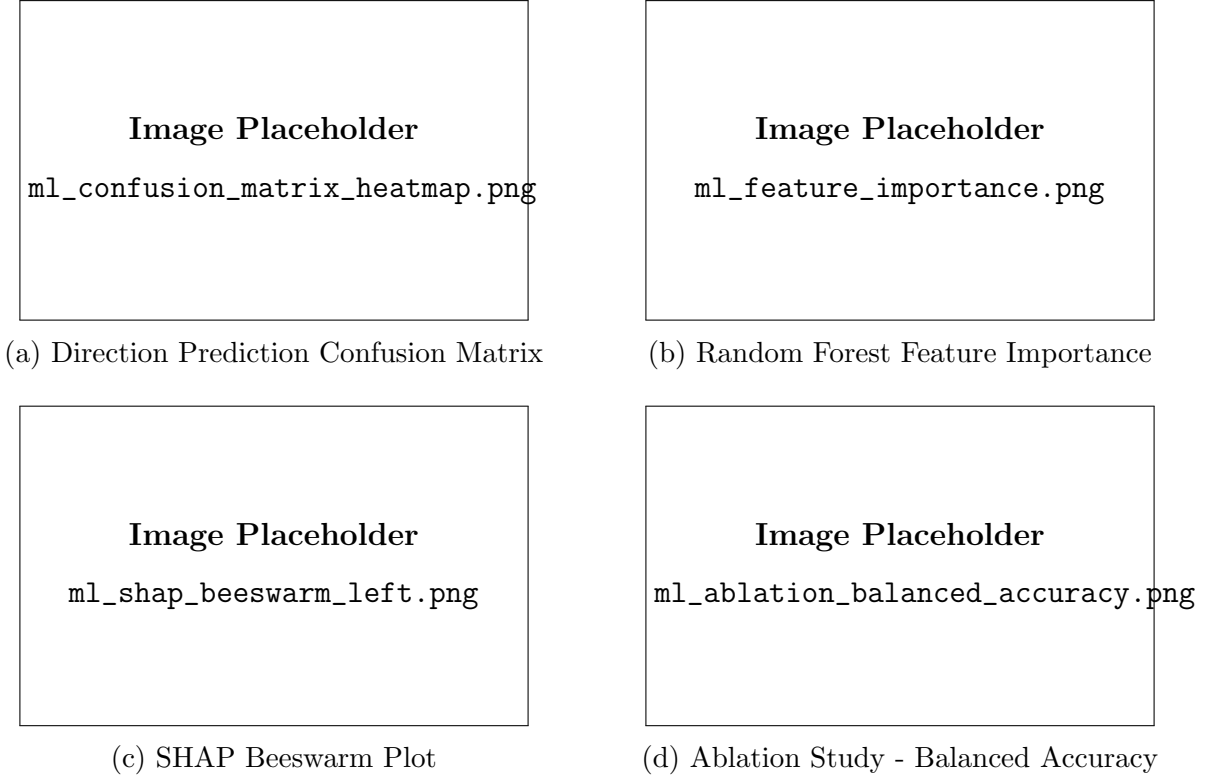


Figure 6: Machine learning model evaluation

namic detection of system compilers, such as `g++`, `clang++`, or `cl`, and builds C++ executables on demand.

5.2 Limitations

Experimental testing reveals certain limitations of the current system at the algorithmic and computational levels. In the chase scenario, the probabilistic nature of the SA algorithm results in highly random behavior, with a capture rate of only 83.3% and a decision latency of 9.99 milliseconds. Although the Minimax algorithm achieves the highest capture efficiency, with an average 20.5-step capture, its depth search tree brings a decision overhead of 0.62 milliseconds, significantly higher than the A* algorithm at 0.0511 milliseconds. This shows that deep adversarial search has a scalability bottleneck when applied to larger maze topology. Furthermore, deterministic algorithms such as A* and Greedy Search rely too heavily on static heuristics, making it difficult for them to adapt to dynamically changing human-player strategies across different sessions.

For predictive modeling, existing frameworks over-rely on absolute spatial coordinates, such as `human_col` and `human_row`, and short-term historical move counts, such as `prev_down.1`. This can lead to state confusion and limit the model’s ability to represent long-term strategic intent. The extraction of environmental features is limited to direct wall constraints, neglecting macro-topological features such as distant dead ends and levels.

At the system integration level, the execution of the C++ algorithm depends on synchronous read and write operations on the temporary text file `map/generated_map.txt`, which can lead to disk I/O bottlenecks and potential race conditions in a multi-client concurrent environment. In addition, the genetic algorithm map generation process is

computationally intensive and can briefly block the main server thread during session initialization, resulting in loading delay.

5.3 Future Work

Future research will focus on incorporating machine learning methods. We plan to improve the feature representation mechanism by utilizing convolutional spatial mapping, which can help extract macro-topological structures and time-series data. Additionally, we will introduce reinforcement learning (RL), particularly Deep Q-Networks (DQN), enabling AI agents to actively adjust their behavioral strategies in real time and reduce reliance on static heuristic functions.

For system architecture optimization, we plan to decouple C++ algorithm execution from disk I/O through direct memory binding or standard stream pipelines. At the network layer, we intend to replace the existing HTTP polling mechanism with WebSockets to establish real-time bidirectional communication, thereby reducing request overhead and supporting synchronous player-versus-player (PvP) modes.

Moreover, we plan to introduce multithreading or precalculated random seeds to refactor the GA algorithm's map generation process, thereby reducing server blocking time. Finally, we will further expand the game mechanism by designing maze generation logic that includes a clear exit or target area, provides victory conditions based on spatial location, and transforms the game goal from simple survival time to a mission-oriented and strategic escape experience.

References

...